

PONTIFICIA UNIVERSIDAD JAVERIANA

FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

LABORATORIO 1

Implementación de Monolito con patrón MVC, DAO y API
REST en .NET 7

Santiago Mesa
Camilo Peñuela
Sergio Orosman Parra

Docente: Andrés Sánchez

Asignatura: Arquitectura de Software

11 de mayo de 2026
Bogotá D.C., Colombia

Índice

1	Marco Conceptual	2
1.1	Arquitectura Monolítica	2
1.2	Patrón MVC (Model-View-Controller)	2
1.3	Patrón DAO / Repository	2
2	Diseño del Sistema	2
2.1	Modelo Relacional	2
2.2	API REST (Swagger)	3
3	Procedimiento y Configuración	4
3.1	Paso 1: Configuración de Docker Compose	4
3.2	Paso 2: Limpieza y Automatización del Script SQL	5
3.3	Paso 3: Corrección de Enrutamiento en Program.cs	5
3.4	Paso 4: Generación de Vistas MVC Faltantes	5
4	Conclusiones	5
5	Referencias	5

1. Marco Conceptual

Este proyecto explora la implementación práctica de patrones de diseño fundamentales en la construcción de aplicaciones empresariales.

1.1. Arquitectura Monolítica

La arquitectura monolítica es un enfoque tradicional en el desarrollo de software donde todos los componentes de una aplicación (interfaz de usuario, lógica de negocio y acceso a datos) se integran y despliegan como una única unidad indivisible. En este laboratorio, se construyó un sistema en .NET 7 que encapsula el consumo de la base de datos y la exposición web en un único paquete ejecutable.

1.2. Patrón MVC (Model-View-Controller)

El patrón MVC separa la aplicación en tres componentes lógicos principales:

- **Modelos:** Representan los datos de la aplicación y la lógica de negocio (implementado con Entity Framework Core).
- **Vistas:** Interfaz visual con la que interactúa el usuario (Archivos `.cshtml` renderizados en el servidor).
- **Controladores:** Procesan las peticiones del usuario, manipulan el modelo y retornan la vista adecuada.

1.3. Patrón DAO / Repository

Para abstraer el acceso a datos y desacoplar la lógica de Entity Framework de los controladores, se utilizó el patrón de repositorio. Se crearon interfaces genéricas (`IPersonaRepository`, `IEstudioRepository`) que encapsulan las operaciones CRUD.

Orquestación con Docker: Un requisito fundamental fue la configuración de todo el ecosistema usando contenedores aislados. Esto asegura que la aplicación y su base de datos corran en un entorno estandarizado sin depender de las configuraciones locales de la máquina anfitriona (evitando conflictos de `localhost`).

2. Diseño del Sistema

2.1. Modelo Relacional

La aplicación administra información de personas y sus relaciones académicas y telefónicas. A continuación, se ilustra la relación entre las entidades principales:

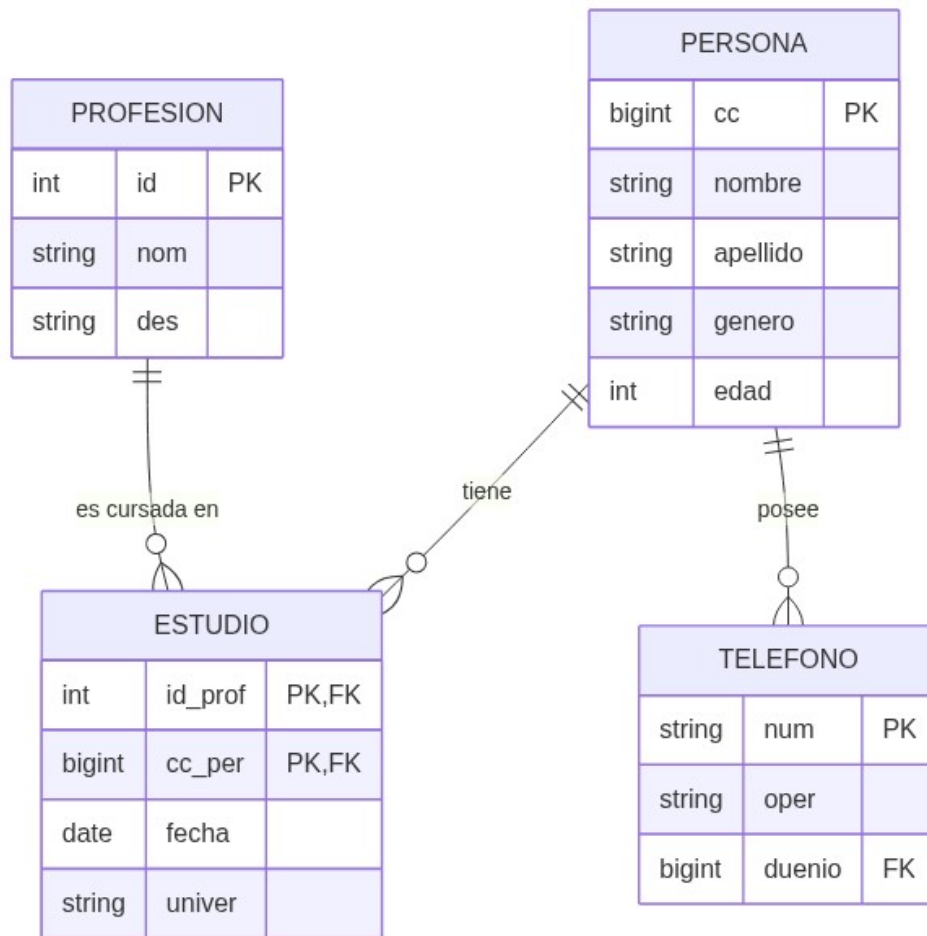


Figura 1: Diagrama Entidad-Relación (Exportado desde Mermaid)

Consta de cuatro tablas principales:

1. **Persona:** Entidad central con identificador (Cédula).
2. **Profesión:** Catálogo de profesiones disponibles.
3. **Estudio:** Entidad puente (llave compuesta) que asocia a una **Persona** con una **Profesión**.
4. **Teléfono:** Datos de contacto de la **Persona**.

2.2. API REST (Swagger)

El monolito no solo renderiza páginas web, sino que expone una capa de servicios REST documentada mediante Swagger. Se implementaron los siguientes controladores API:

- /api/Persona
- /api/Estudio
- /api/Profesion
- /api/Telefono

3. Procedimiento y Configuración

Para garantizar la correcta ejecución multiplataforma (compatible con macOS y Windows) y resolver los retos del laboratorio, se configuró una arquitectura de contenedores usando Docker.

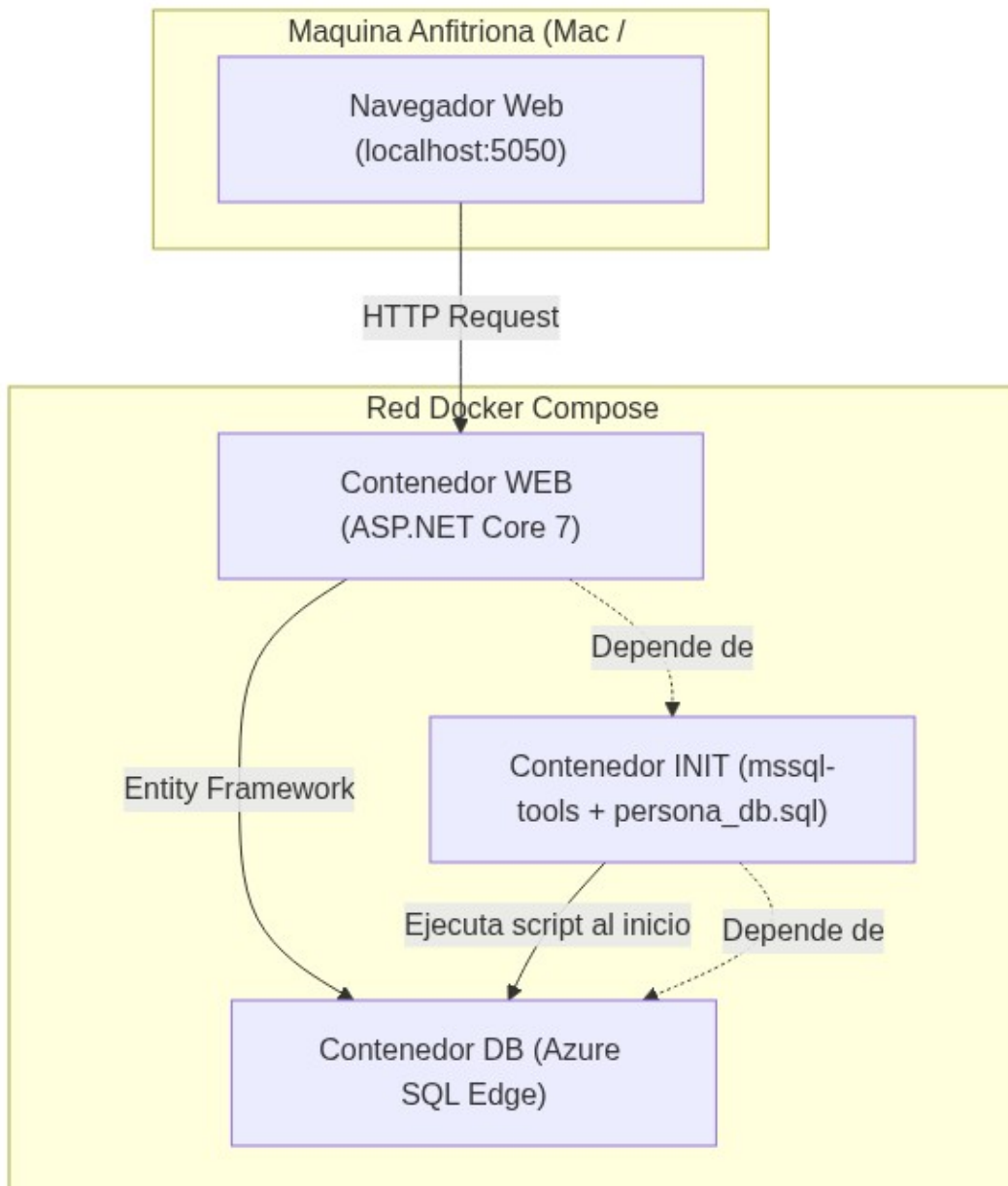


Figura 2: Arquitectura de Orquestación Docker (Exportado desde Mermaid)

3.1. Paso 1: Configuración de Docker Compose

Se eliminó la dependencia de "localhost" del entorno local creando un `docker-compose.yml`. Se configuraron tres contenedores clave:

- `sql_server_db`: Contenedor basado en `azure-sql-edge` que provee el motor de SQL Server.
- `db_init`: Un contenedor temporal basado en `mssql-tools` que espera a que el servidor SQL encienda y ejecuta automáticamente el script `persona_db.sql` para inyectar las tablas y datos.

- **web:** La aplicación ASP.NET, inyectándole la variable de entorno `ConnectionStrings__DefaultConnection` apuntando al contenedor de la base de datos.

3.2. Paso 2: Limpieza y Automatización del Script SQL

El script original presentaba errores de sintaxis y conflictos de caracteres CRLF de Windows que causaban que Docker en Mac fallara al inicializar las tablas (Error 500 en la API). Se reescribió el archivo `persona_db.sql` aplicando limpieza de caracteres y agregando instrucciones explícitas de término (GO y QUIT) para automatizar el llenado masivo sin interacción manual.

3.3. Paso 3: Corrección de Enrutamiento en Program.cs

Originalmente, el proyecto únicamente resolvía rutas para la API REST debido a la instrucción `app.MapControllers()`. Se agregó la lógica fundamental de enrutamiento MVC tradicional para que el sistema pudiera ubicar y servir las pantallas web de los usuarios:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

3.4. Paso 4: Generación de Vistas MVC Faltantes

Para cumplir completamente con el patrón MVC, se crearon los archivos `.cshtml` (vistas basadas en Bootstrap 5) agrupados por controlador. Al crear directorios como `Views/Estudio` o `Views/Telefono` se solucionaron múltiples errores de renderizado. Ahora, cada tabla cuenta con su respectivo listado (*Index*), creación (*Create*), edición (*Edit*), borrado (*Delete*) y detalles (*Details*).

4. Conclusiones

- **Aislamiento Efectivo:** Utilizar Docker elimina la fricción entre sistemas operativos (macOS vs Windows). Al empaquetar SQL Server y .NET en contenedores mediante `docker-compose`, se asegura que el entorno sea idéntico para cualquier desarrollador y profesor que evalúe el código.
- **Híbrido MVC-API:** Este laboratorio demostró cómo .NET permite coexistir de forma eficiente un backend tipo API REST (`[ApiController]`) con una aplicación web completa orientada a vistas en el mismo monolito.
- **Convención sobre Configuración:** La corrección del enrutamiento demostró la importancia de respetar las convenciones de ASP.NET Core, donde las vistas deben residir estrictamente en carpetas nombradas idénticamente al controlador que las solicita.

5. Referencias

- Microsoft Documentation. (2023). *Tutorial: Create a web API with ASP.NET Core*.
- Guía del Laboratorio 1 (DotNet) proporcionada por la asignatura.
- Fowler, M. (2015). *Microservices and Monoliths*.